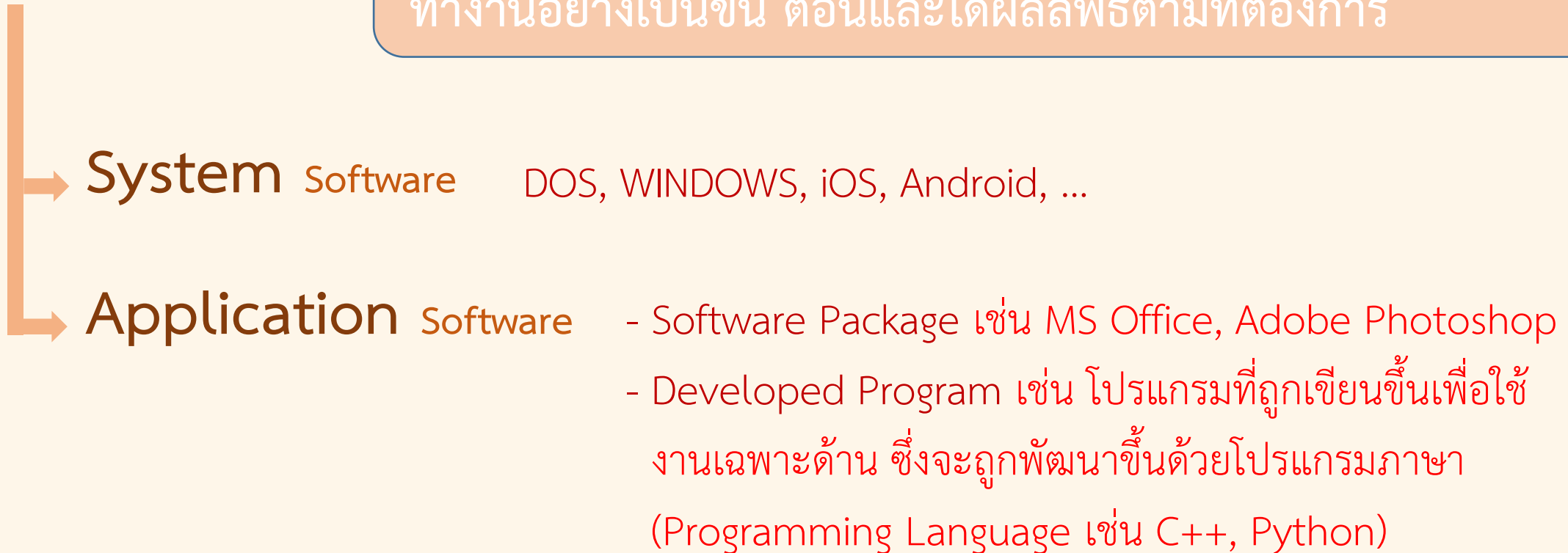




พื้นฐานระบบคอมพิวเตอร์ และ การออกแบบโปรแกรม

Software

คือ โปรแกรม หรือ ชุดคำสั่งที่โปรแกรมเมอร์สร้างขึ้นเพื่อให้คอมพิวเตอร์ทำงานอย่างเป็นขั้น ตอนและได้ผลลัพธ์ตามที่ต้องการ





Programming Language : โปรแกรมภาษา

ใช้ในการพัฒนาโปรแกรมประยุกต์สำหรับงานเฉพาะตามที่ใช้ต้องการ

ประเภทของโปรแกรมภาษา

1. ภาษาระดับต่ำ (Low-Level Language)

ได้แก่ ภาษาเครื่อง (Machine Language)

2. ภาษาระดับกลาง (Middle-Level Language)

ได้แก่ ภาษาแอสเซมบลี (Assembly Language)

3. ภาษาระดับสูง (High-Level Language)

ได้แก่ C++, Java, Python, ...

Example of machine language instruction

0110	0011	0010	0001
0100	0011	0010	0010
0100	0011	0010	0011
0111	0011	0010	0100



Programming Language : โปรแกรมภาษา

ใช้ในการพัฒนาโปรแกรมประยุกต์สำหรับงานเฉพาะตามที่ใช้ต้องการ

ประเภทของโปรแกรมภาษา

1. ภาษาระดับต่ำ (Low-Level Language)

ได้แก่ ภาษาเครื่อง (Machine Language)

2. ภาษาระดับกลาง (Middle-Level Language)

ได้แก่ ภาษาแอสเซมบลี (Assembly Language)

3. ภาษาระดับสูง (High-Level Language)

ได้แก่ C++, Java, Python, ...

Example of assembly language instruction

LDA X	01100011	00100001
ADA Y	01000011	00100010
ADA Z	01000011	00100011
STA D	01110011	00100100

Assembler





Programming Language : โปรแกรมภาษา

ใช้ในการพัฒนาโปรแกรมประยุกต์สำหรับงานเฉพาะตามที่ใช้ต้องการ

ประเภทของโปรแกรมภาษา

1. ภาษาระดับต่ำ (Low-Level Language)

ได้แก่ ภาษาเครื่อง (Machine Language)

2. ภาษาระดับกลาง (Middle-Level Language)

ได้แก่ ภาษาแอสเซมบลี (Assembly Language)

3. ภาษาระดับสูง (High-Level Language)

ได้แก่ C++, Java, Python, ...

Example of high-level language instruction

C++ language instruction

$D = X + Y + Z$

Compiler

LDA X

ADA Y

ADA Z

STA D

Assembler

01100011 00100001

01000011 00100010

01000011 00100011

01110011 00100100



Programming Language : โปรแกรมภาษา

ตัวแปลภาษา ทำหน้าที่แปลรหัสคำสั่งจากภาษาระดับสูงให้เป็นรูปแบบภาษาเครื่อง มี 2 ลักษณะ คือ

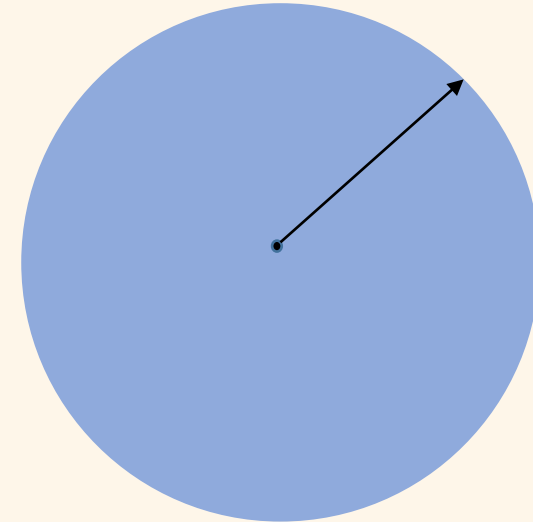
ตัวแปลภาษา	ข้อดี	ข้อเสีย
คอมไพเลอร์ (Compiler)	หากโปรแกรมทำการแปลรหัสคำสั่งผ่านแล้วไม่เป็น ต้องทำการแปลรหัสคำสั่งซ้ำอีก ทำให้การทำงานใน ครั้งต่อไปทำได้รวดเร็ว	เมื่อเกิดข้อผิดพลาดจะทำการตรวจสอบหาได้ยาก
อินเตอร์พรีเตอร์ (Interpreter)	- หากมีข้อผิดพลาดสามารถตรวจสอบได้ง่าย - สามารถทำงานเฉพาะจุดได้ ไม่ต้องเสียเวลารอการ แปลรหัสทั้งหมดของโปรแกรม	- หากต้องการทำงานตามคำสั่งของโปรแกรม ทั้งหมดจะต้องทำการแปลรหัสคำสั่งที่ละบรรทัดทำ ให้ทำงานช้า - ต้องทำการแปลรหัสคำสั่งทุกครั้งที่ใช้งานโปรแกรม

การพัฒนาโปรแกรมประกอบด้วยขั้นตอนหลัก 6 ขั้นตอน

1. การวิเคราะห์ปัญหา
2. การออกแบบโปรแกรม
3. การเขียนโปรแกรมด้วยภาษาคอมพิวเตอร์
4. การทดสอบและแก้ไขโปรแกรม
5. การทำเอกสารประกอบโปรแกรม
6. การบำรุงรักษาโปรแกรม



โจทย์ : จงพัฒนาโปรแกรมหาพื้นที่
วงกลมโดยรับค่ารัศมีของผู้ใช้
แล้วแสดงผลลัพธ์ออก
ทางจอภาพ

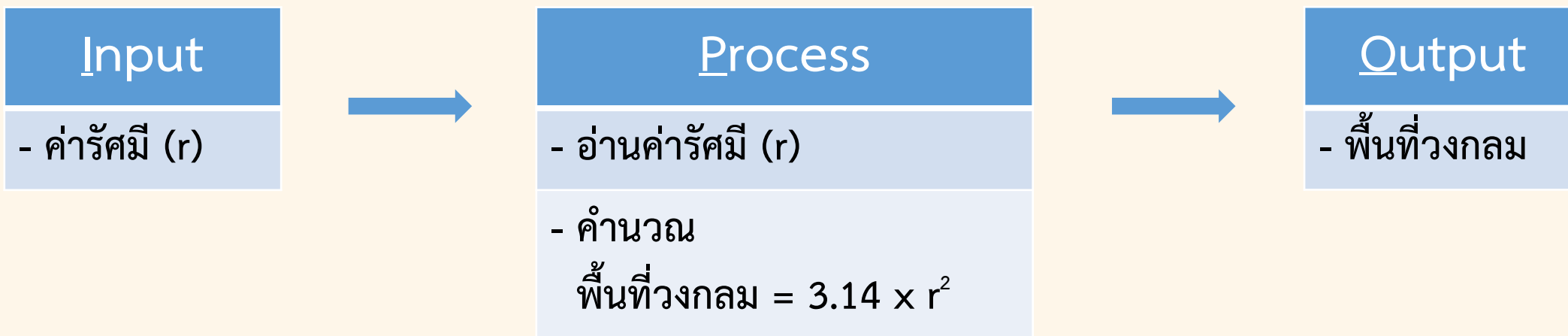


1. การวิเคราะห์ปัญหา (Problem Analysis)

- ผู้พัฒนาโปรแกรมต้องวิเคราะห์ปัญหาว่าต้องเขียนโปรแกรมเพื่อแก้ไขปัญหอะไร
- หาแนวทางการแก้ไขปัญหที่เหมาะสม เพื่อให้การทำงานมีประสิทธิภาพ
- นำ IPO Analysis มาใช้ (Input > Process > Output)



โจทย์ : จงพัฒนาโปรแกรมหาพื้นที่วงกลมโดยรับค่ารัศมีของผู้ใช้
แล้วแสดงผลลัพธ์ออกทางจอภาพ



“เมื่อนักเรียนมีเงินจำนวน 500 บาท
เข้าไปซื้อสินค้าในร้านค้าหน้าโรงเรียน
นักเรียนมีวิธีการคำนวณราคาสินค้า
และชำระเงินอย่างไร”

ให้นักเรียนระบุ input process output





“จากสถานการณ์ในปัจจุบันการขายของออนไลน์ มีบทบาทเข้ามาในชีวิตประจำวันของเราเป็นอย่างมาก ร้านค้าออนไลน์มักจะมีโปรโมชั่นเพื่อกระตุ้นยอดขาย คือการมีส่วนลดในวันพิเศษต่าง ๆ เช่น โปรโมชั่น 11.11 , 5.5 อื่นๆ เมื่อเราจะซื้อสินค้ามากกว่า 1 ชิ้น แล้วได้ส่วนลดนั้น นักเรียนมีความคิดเห็นอย่างไรจากการกำหนดโปรโมชั่นเหล่านี้ และหากเราต้องการคำนวณราคาสินค้าจากโปรโมชั่นเราวิธีการคิดอย่างไร”

ให้นักเรียนระบุ input process output



2. การออกแบบ (Design)

- วางแผนขั้นตอนว่าจะต้องเขียนโปรแกรมอย่างไร
- ลำดับขั้นตอนการทำงานของโปรแกรมเรียกว่า อัลกอริทึม (Algorithm)
โดยสามารถแสดงขั้นตอนการทำงานได้ 2 ลักษณะ คือ
 - 1) Pseudo Code เป็นการเขียนขั้นตอนการทำงานของโปรแกรมด้วยภาษาอย่างง่าย ๆ โดยไม่ขึ้นกับภาษาคอมพิวเตอร์ใด
 - 2) Flowchart เป็นการเขียนขั้นตอนการทำงานโดยใช้สัญลักษณ์ที่กำหนดไว้เป็นมาตรฐานสากล



2. การออกแบบ (Design)

- การเขียน Pseudo Code สำหรับปฏิบัติการทางคอมพิวเตอร์ขั้นพื้นฐาน 6 ประการ

1) การรับข้อมูล ประกอบด้วย

- READ ใช้เพื่ออ่านข้อมูลจากแฟ้มข้อมูล
- GET, INPUT ใช้เพื่อรับข้อมูลจากแป้นพิมพ์

Ex. **READ id, name, class**

INPUT num_1, num_2



2. การออกแบบ (Design)

- การเขียน Pseudo Code สำหรับปฏิบัติการทางคอมพิวเตอร์ขั้นพื้นฐาน 6 ประการ

2) การแสดงข้อมูล ประกอบด้วย

- PRINT ใช้เพื่อแสดงผลข้อมูลออกทางเครื่องพิมพ์
- WRITE ใช้เพื่อเขียนข้อมูลลงในแฟ้มข้อมูล
- DISPLAY, OUTPUT ใช้เพื่อแสดงข้อมูลทางจอภาพ

Ex. **PRINT “Program Completed”**

OUTPUT num_1, num_2



2. การออกแบบ (Design)

- การเขียน Pseudo Code สำหรับปฏิบัติการทางคอมพิวเตอร์ขั้นพื้นฐาน 6 ประการ

3) การคำนวณ คือ การใช้สัญลักษณ์ตัวดำเนินการต่างๆ สำหรับการคำนวณประกอบด้วย
สัญลักษณ์ +, -, *, /, (), %

Ex. $\text{area} = 3.14 * r * r$

$\text{ans} = n \% 2$

$\text{total} = a * (b + c)$

2. การออกแบบ (Design)

- การเขียน Pseudo Code สำหรับปฏิบัติการทางคอมพิวเตอร์ขั้นพื้นฐาน 6 ประการ

4) การกำหนดค่า คือ มี 2 ลักษณะ ประกอบด้วย

- SET ใช้เพื่อกำหนดค่าเริ่มต้นให้กับตัวแปร
- สัญลักษณ์ = ใช้เพื่อกำหนดค่าที่ได้จากการประมวลผล

Ex. **SET n to 10**

sum = sum + n

2. การออกแบบ (Design)

- การเขียน Pseudo Code สำหรับปฏิบัติการทางคอมพิวเตอร์ขั้นพื้นฐาน 6 ประการ

5) การเปรียบเทียบ การกำหนดทางเลือก เงื่อนไข ประกอบด้วย

- IF ใช้เพื่อเริ่มต้นการกำหนดเงื่อนไข
- THEN ใช้เพื่อกำหนดคำสั่งที่ต้องการให้กระทำเมื่อเงื่อนไขเป็นจริง
- ELSE ใช้เพื่อกำหนดคำสั่งที่ต้องการให้กระทำเมื่อเงื่อนไขเป็นเท็จ

2. การออกแบบ (Design)

- การเขียน Pseudo Code สำหรับปฏิบัติการทางคอมพิวเตอร์ขั้นพื้นฐาน 6 ประการ

5) การเปรียบเทียบ การกำหนดทางเลือกเงื่อนไข

```
Ex.      IF score >= 50 THEN  
          DISPLAY "Pass"  
  
        ELSE  
          DISPLAY "Failure"  
  
        ENDIF
```

2. การออกแบบ (Design)

- การเขียน Pseudo Code สำหรับปฏิบัติการทางคอมพิวเตอร์ขั้นพื้นฐาน 6 ประการ

6) การทำงานซ้ำขั้นตอนเดิม ประกอบด้วย

- **WHILE .. ENDWHILE** ใช้เพื่อกำหนดการกระทำซ้ำโดยจะตรวจสอบเงื่อนไขก่อนทำงานในลูป
- **REPEAT .. UNTIL** ใช้เพื่อกำหนดการกระทำซ้ำโดยจะทำคำสั่งในลูปก่อน 1 ครั้ง ก่อนตรวจสอบเงื่อนไข
- **FOR .. NEXT** ใช้เพื่อกำหนดการกระทำซ้ำโดยระบุจำนวนรอบการทำงานในลูปที่แน่นอน



2. การออกแบบ (Design)

- การเขียน Pseudo Code สำหรับปฏิบัติการทางคอมพิวเตอร์ขั้นพื้นฐาน 6 ประการ

6) การทำงานซ้ำขั้นตอนเดิม แบบ **WHILE .. ENDWHILE**

Ex. **SET 1 to n**

WHILE n <= 10

INPUT id, name, gpa

OUTPUT id, name, gpa

ADD 1 to n

ENDWHILE



2. การออกแบบ (Design)

- การเขียน Pseudo Code สำหรับปฏิบัติการทางคอมพิวเตอร์ขั้นพื้นฐาน 6 ประการ

6) การทำงานซ้ำขั้นตอนเดิม แบบ **REPEAT .. UNTIL**

Ex. **OPEN student_file**

SET 0 to total

REPEAT

READ id, name, score

ADD score to total

UNTIL end of file

OUTPUT total



โจทย์ : จงพัฒนาโปรแกรมหาพื้นที่วงกลม โดยรับค่ารัศมีของผู้ใช้แล้วแสดงผลลัพธ์
ออกทางจอภาพ

Pseudo Code

START

INPUT r

area = $3.14 * r * r$

OUTPUT area

STOP

2. การออกแบบ (Design)

- การเขียน Flowchart

สัญลักษณ์พื้นฐานที่ใช้ในการเขียน Flowchart



(Terminal)

จุดเริ่มต้น, จุดสิ้นสุดการทำงาน



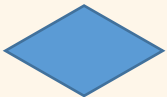
(Input / Output)

รับข้อมูล, แสดงผลข้อมูล



(Process)

การคำนวณ, การกำหนดค่า



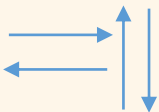
(Decision)

การตัดสินใจ, เงื่อนไข



(Connector)

จุดเชื่อมต่อ



(Flow Line)

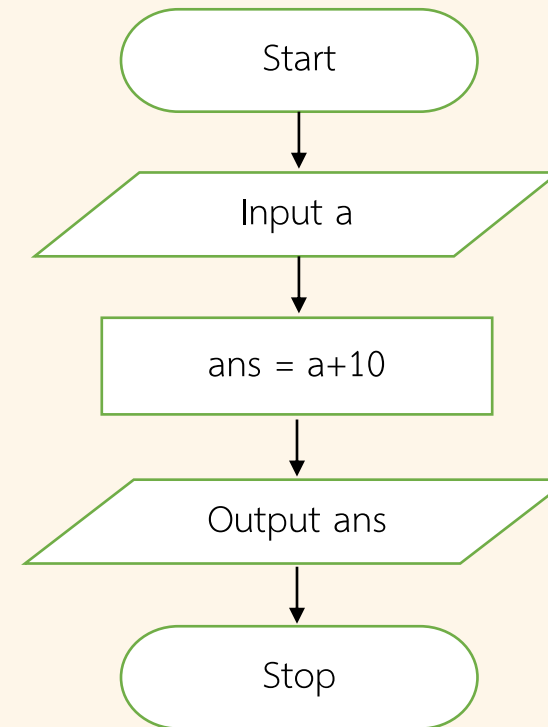
ทิศทางการทำงาน

2. การออกแบบ (Design)

- การเขียน Flowchart

โครงสร้างในการเขียน Flowchart

➤ โครงสร้างแบบลำดับ
(Sequence Structure)

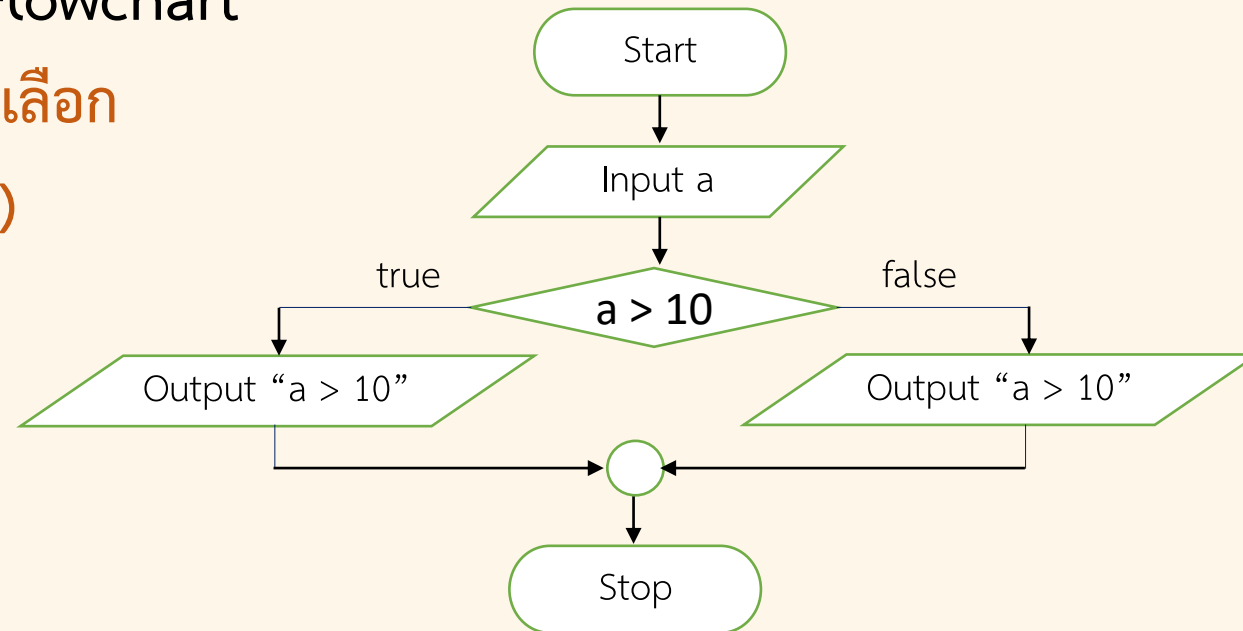


2. การออกแบบ (Design)

- การเขียน Flowchart

โครงสร้างในการเขียน Flowchart

➤ โครงสร้างแบบทางเลือก
(Selection Structure)

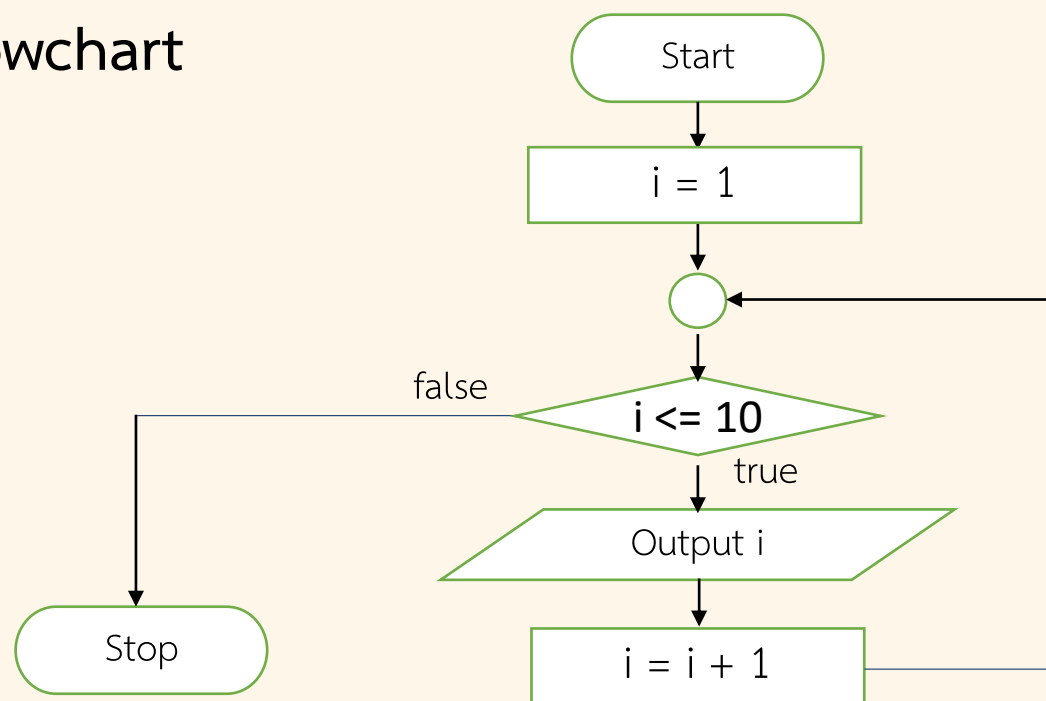


2. การออกแบบ (Design)

- การเขียน Flowchart

โครงสร้างในการเขียน Flowchart

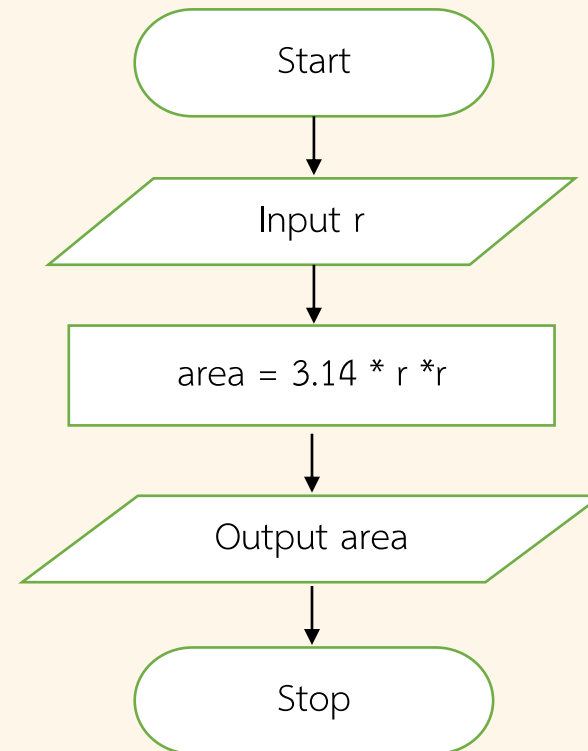
➤ โครงสร้างแบบทำซ้ำ
(Repetition Structure)





โจทย์ : จงพัฒนาโปรแกรมหาพื้นที่วงกลม โดยรับค่ารัศมีของผู้ใช้แล้วแสดงผลลัพธ์ออกทางจอภาพ

Flowchart





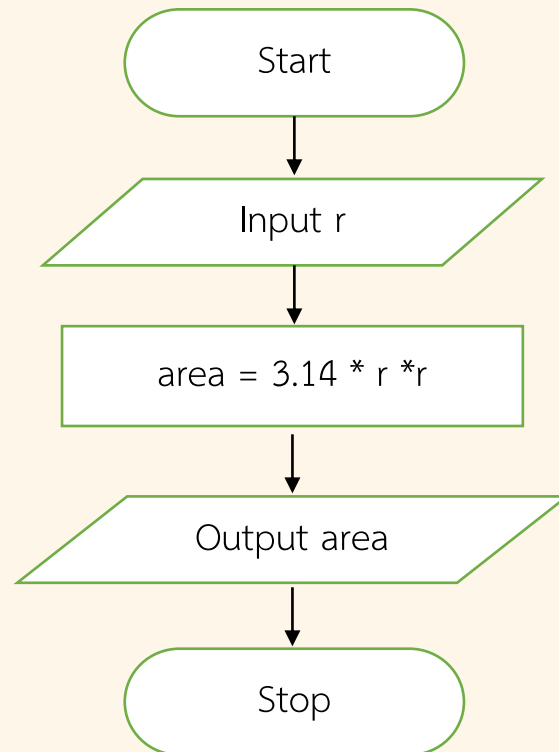
3. การเขียนโปรแกรม (Coding)

- นำอัลกอริทึมที่ออกแบบไว้มาเขียนโปรแกรมตามหลักไวยากรณ์ (Syntax) ของภาษาคอมพิวเตอร์ที่เลือกใช้



โจทย์ : จงพัฒนาโปรแกรมหาพื้นที่วงกลม โดยรับค่ารัศมีของผู้ใช้แล้วแสดงผลลัพธ์ออกทางจอภาพ

Flowchart



Coding

```
#include <stdio.h>
#include <conio.h>
main()
{
    int r ;
    float area ;
    scanf("%d",&r) ;
    area = 3.14 * r * r ;
    printf("%f",area) ;
    getch() ;
}
```

4. การทดสอบและแก้ไขข้อผิดพลาด (Testing & Debugging)

- หลังจากเขียนโปรแกรมจะต้องทำการทดสอบความถูกต้องของโปรแกรม
- หาจุดผิดพลาดของโปรแกรม (Bug)
- การแก้ไขจุดผิดพลาดของโปรแกรมเรียกว่า ดีบั๊ก (Debug)
- ข้อผิดพลาดของโปรแกรมที่พบบ่อย มี 2 ลักษณะ
 - 1) Syntax Error คือการเขียนโปรแกรมผิดไวยากรณ์ เช่น ลืมใส่ ; ปิดท้ายคำสั่ง
 - 2) Logic Error คือการเขียนขั้นตอนคำสั่งในการคำนวณผลลัพธ์ผิด

5. การทำเอกสารคู่มือ (Documentation)

- การจัดทำคู่มือการใช้งานโปรแกรมจะทำให้ผู้ใช้งานโปรแกรมได้อย่างถูกต้องและมีประสิทธิภาพ โดยมีลักษณะคู่มือ 2 ลักษณะ
 - 1) User Document หรือ User Guide เป็นการอธิบายการใช้งานโปรแกรมสำหรับผู้ใช้งาน
 - 2) Programmer Document เป็นเอกสารสำหรับให้ข้อมูลเพื่อทำการแก้ไขและพัฒนาโปรแกรมต่อไปได้อย่างสะดวก

6. การบำรุงรักษาโปรแกรม (Program Maintenance)

- เมื่อโปรแกรมผ่านการตรวจสอบตามขั้นตอนเรียบร้อยแล้ว และถูกนำมาให้ผู้ใช้ได้ใช้งาน ในช่วงแรก ผู้ใช้อาจจะยังไม่คุ้นเคยก็อาจทำให้เกิดปัญหาขึ้นมาบ้าง ดังนั้นจึงต้องมีผู้คอยควบคุมดูแลและคอยตรวจสอบการทำงาน การบำรุงรักษาโปรแกรมจึงเป็นขั้นตอนที่ผู้เขียนโปรแกรมต้องคอยเฝ้าดูและหาข้อผิดพลาดของโปรแกรมในระหว่างที่ผู้ใช้ใช้งานโปรแกรม และปรับปรุงโปรแกรมเมื่อเกิดข้อผิดพลาดขึ้น หรือในการใช้งานโปรแกรมไปนานๆ ผู้ใช้อาจต้องการเปลี่ยนแปลงการทำงาน ของระบบงานเดิมเพื่อให้เหมาะกับเหตุการณ์ นักเขียนโปรแกรมก็จะต้องคอยปรับปรุงแก้ไขโปรแกรมตามความต้องการของ ผู้ใช้ที่เปลี่ยนแปลงไปนั่นเอง



1. จงเขียน Pseudo Code และ Flowchart เพื่อรับค่าจำนวน 3 จำนวน จากนั้นคำนวณหาค่าเฉลี่ยและแสดงผลลัพธ์ทางจอภาพ

Pseudo Code

START

SET 0 to num_1, 0 to num_2,
0 to num_3, 0 to avg

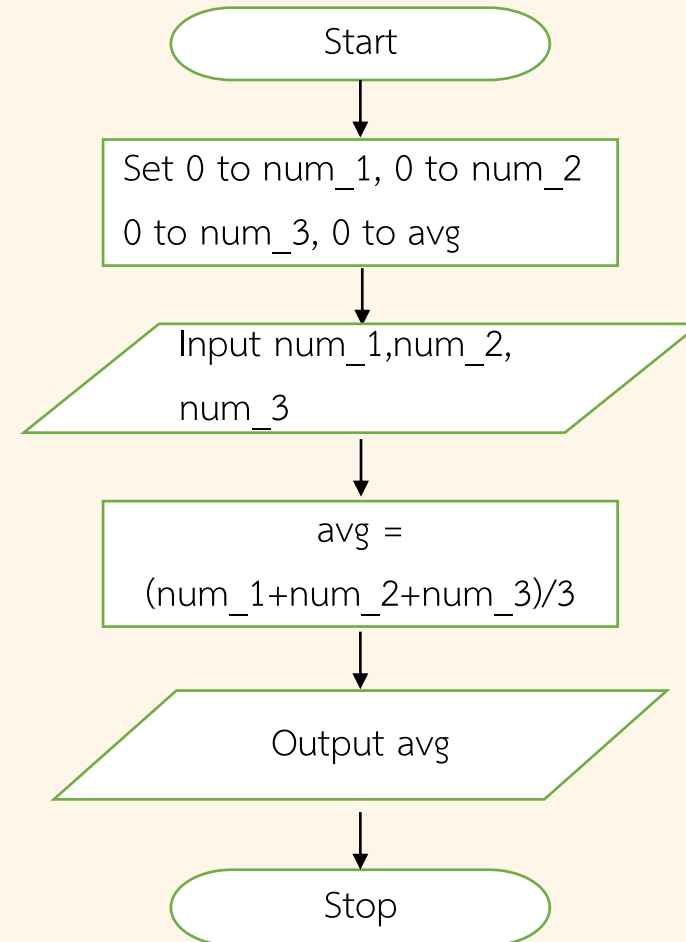
INPUT num_1, num_2, num_3

avg = (num_1 + num_2 + num_3) / 3

DISPLAY avg

STOP

Flowchart



2. จงเขียน Pseudo Code และ Flowchart เพื่อรับค่าเก็บในตัวแปร 2 ตัว ดังนี้ จำนวนที่ 1 เก็บในตัวแปร a, จำนวนที่ 2 เก็บในตัวแปร b จากนั้น ให้เปรียบเทียบค่าของตัวแปรทั้ง 2 ตัว ถ้าตัวแปร a มีค่ามากกว่าให้แสดงข้อความ $a > b$ ถ้า ตัวแปร b มีค่ามากกว่าให้แสดงข้อความ $a < b$



2. จงเขียน Pseudo Code และ Flowchart เพื่อรับค่าเก็บในตัวแปร 2 ตัว ดังนี้ จำนวนที่ 1 เก็บในตัวแปร a, จำนวนที่ 2 เก็บในตัวแปร b จากนั้น ให้เปรียบเทียบค่าของตัวแปรทั้ง 2 ตัว ถ้าตัวแปร a มีค่ามากกว่าให้แสดงข้อความ $a > b$ ถ้า ตัวแปร b มีค่ามากกว่าให้แสดงข้อความ $a < b$

Pseudo Code

START

SET 0 to a, 0 to b

INPUT a, b

IF $a > b$ THEN

DISPLAY " $a > b$ "

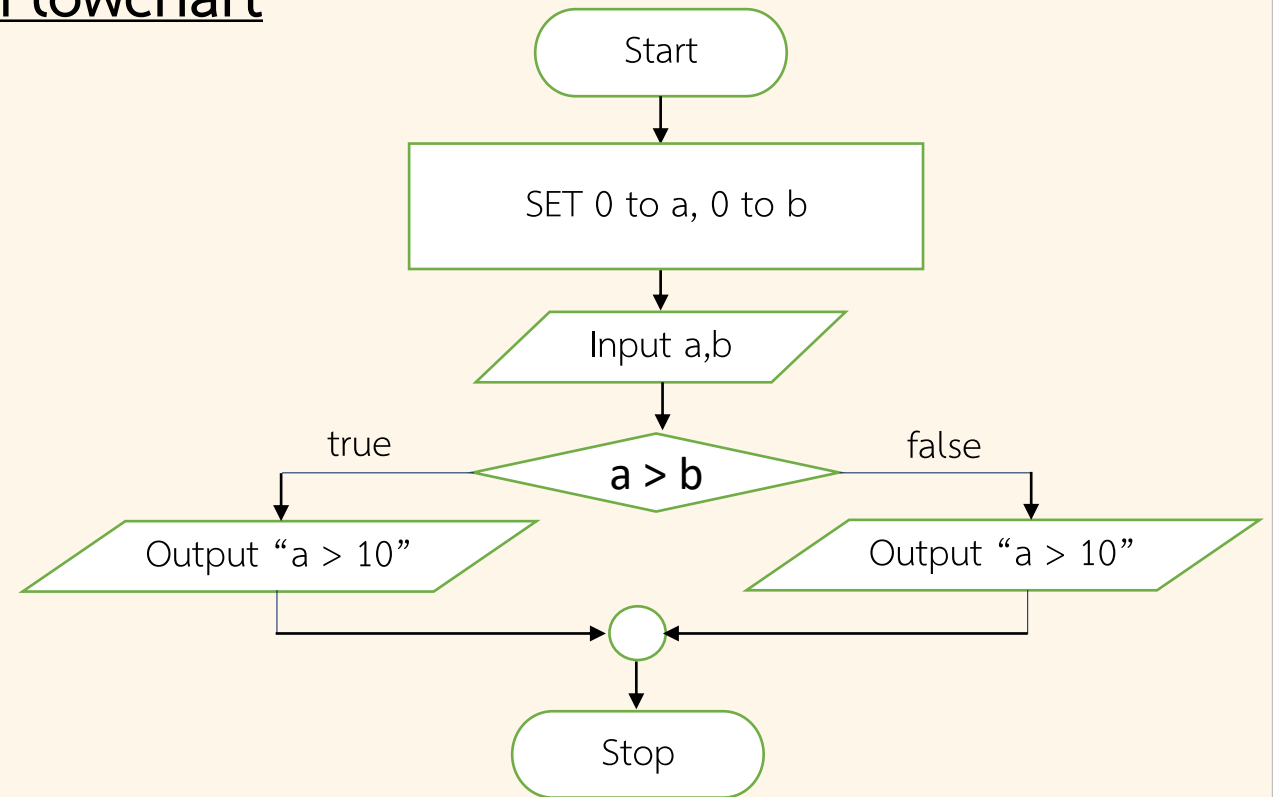
ELSE

DISPLAY " $a < b$ "

ENDIF

STOP

Flowchart



3. จงเขียน Pseudo Code และ Flowchart เพื่อรับค่าอุณหภูมิเก็บในตัวแปร t จากนั้น ทำการเปรียบเทียบค่าอุณหภูมิตามเงื่อนไขต่อไปนี้
- ถ้าค่าอุณหภูมิต่ำกว่า 16 ให้แสดงข้อความ “Cool”
 - ถ้าค่าอุณหภูมิตั้งแต่ 16 ถึง 25 ให้แสดงข้อความ “Warm”
 - ถ้าค่าอุณหภูมิตั้งแต่ 26 ขึ้นไป ให้แสดงข้อความ “Hot”



4. จงเขียน Pseudo Code และ Flowchart เพื่อรับค่าจำนวนเต็มเก็บในตัวแปร n จากนั้น ให้โปรแกรมแสดงค่าตัวเลขเริ่มตั้งแต่ 0 ถึง จำนวนที่เก็บในตัวแปร n

5. จงเขียน Pseudo Code และ Flowchart เพื่อรับคะแนนนักเรียนเก็บในตัวแปร score จากนั้น ทำการเปรียบเทียบคะแนนตามเงื่อนไขต่อไปนี้
- ถ้าคะแนนต่ำกว่า 50 ให้แสดงข้อความ “Failure”
 - ถ้าคะแนนตั้งแต่ 50 ถึง 69 ให้แสดงข้อความ “Pass”
 - ถ้าคะแนนตั้งแต่ 70 ถึง 100 ให้แสดงข้อความ “Good”
 - ถ้าคะแนนไม่อยู่ในช่วง 0 ถึง 100 ให้แสดงข้อความ “Score Error”

6. จงเขียน Pseudo Code และ Flowchart เพื่อรับค่าจำนวนเต็มเก็บในตัวแปร n ถ้าค่า n ไม่อยู่ในช่วง 1 ถึง 100 ให้แสดงข้อความ “Number Over” แล้วรับค่า n ใหม่ จากนั้น ให้โปรแกรมแสดงค่าตัวเลขเฉพาะเลขคู่ เริ่มตั้งแต่ 0 ถึง จำนวนที่เก็บในตัวแปร n



7. จงเขียน Pseudo Code และ Flowchart เพื่อหาจำนวนเฉพาะ โดยการรับค่าจากแป้นพิมพ์เป็นจำนวนสูงสุดที่จะหาจำนวนเฉพาะ จากนั้นแสดงผลทางหน้าจอ ดังนี้

Enter number : 20 (กดปุ่ม Enter)

Output : 2, 3, 5, 7, 11, 13, 17, 19

(แสดงจำนวนเฉพาะทั้งหมดที่ไม่เกิน 20)

6. จงเขียน Pseudo Code และ Flowchart เพื่อรับค่าจำนวนเต็มเก็บในตัวแปร n ถ้าค่า n ไม่อยู่ในช่วง 1 ถึง 100 ให้แสดงข้อความ “Number Over” แล้วรับค่า n ใหม่ จากนั้น ให้โปรแกรมแสดงค่าตัวเลขเฉพาะเลขคู่ เริ่มตั้งแต่ 0 ถึง จำนวนที่เก็บในตัวแปร n

8. จงเขียน Pseudo Code และ Flowchart โดยการรับข้อมูลเลขจำนวนเต็ม 3 จำนวน จากนั้น ให้ทำการจัดเรียงจำนวนทั้ง 3 จำนวน จากน้อยไปมากโดยแสดงผลทางหน้าจอ ดังนี้

INPUT

Enter first value : 20 (กดปุ่ม Enter)

Enter second value : 25 (กดปุ่ม Enter)

Enter third value : 19 (กดปุ่ม Enter)

OUTPUT

19 20 25